

ชื่อโครงการ GardenVerse

จัดทำโดย

กลุ่ม GardenGardon

	ชื่อ – นามสกุล	เลขประจำตัว
1.	ณัชนสา เลิศมหากุล	65340500021
2.	นันท์นภัส นันทพรนิรชา	65340500034
3.	อนวัช อนุสุเรนทร์	65340500056
4.	ชญัญญากัฏ ทรัพย์สวัสดิ์กุล	65340500067

FRA502 Web Programming

สถาบันวิทยาการหุ่นยนต์ภาคสนาม

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ภาคการเรียนปลายปีการศึกษา 2569

1. ข้อมูลทั่วไป (บทนำ)

ชื่อโปรเจก (Project Title): GardenVerse

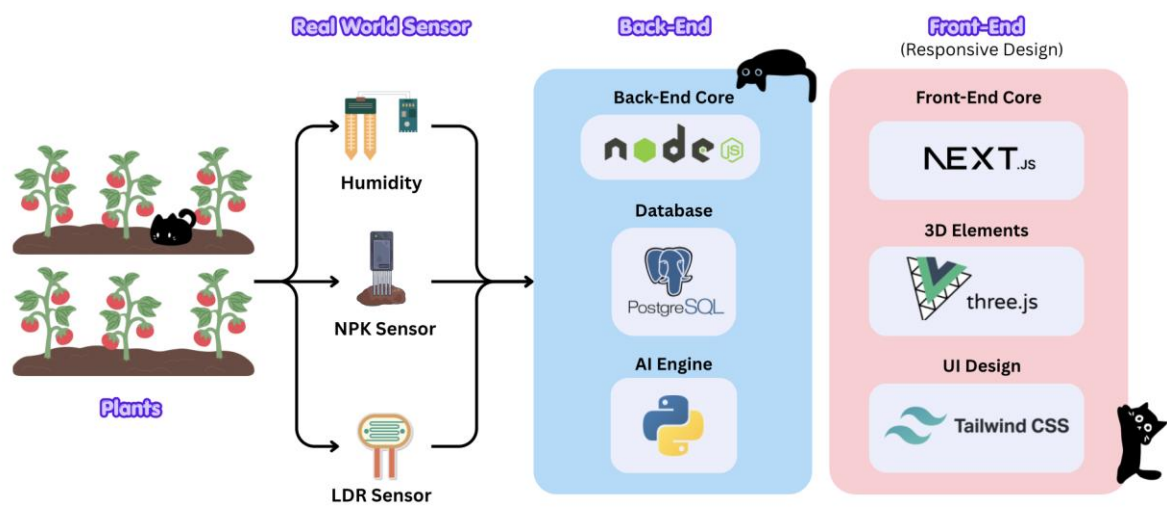
บทคัดย่อ (Abstract):

ในปัจจุบัน การปลูกพืชทั้งเพื่อการบริโภคและการตกแต่งได้รับความนิยมเพิ่มขึ้นอย่างต่อเนื่อง อย่างไรก็ตาม ผู้ปลูกพืชจำนวนมากยังประสบปัญหาในการทำความเข้าใจสภาวะแวดล้อมที่เหมาะสมต่อการเจริญเติบโตของพืช เนื่องจากขาดข้อมูลเกี่ยวกับปัจจัยสำคัญ เช่น ความชื้นในดิน ปริมาณแสงแดด และระดับสารอาหาร ส่งผลให้เกิดการดูแลที่ไม่เหมาะสมและการใช้ทรัพยากรอย่างไม่มีประสิทธิภาพ โครงการนี้มีวัตถุประสงค์เพื่อพัฒนา Web Application ซึ่งได้ตอบสำหรับแสดงผลและวิเคราะห์ข้อมูลสภาพแวดล้อมของพืช โดยมุ่งเน้นการออกแบบระบบที่สามารถรับข้อมูลจากแหล่งภายนอก จัดเก็บ ประมวลผล และนำเสนอผ่านอินเทอร์เฟซบนเว็บในรูปแบบ Dashboard และ Data Visualization ที่เข้าใจง่าย พร้อมทั้งมีการใช้เทคโนโลยีการแสดงผลแบบสามมิติ (3D Visualization) เพื่อจำลองลักษณะของพืชและแสดงสถานะการเจริญเติบโตในรูปแบบที่เสมือนจริงมากยิ่งขึ้น ระบบประกอบด้วยส่วน Frontend สำหรับการแสดงผลและโต้ตอบกับผู้ใช้ และ Backend สำหรับการจัดการข้อมูลและเชื่อมต่อกับแหล่งข้อมูล โดยสามารถแสดงข้อมูลย้อนหลัง วิเคราะห์แนวโน้ม และแจ้งเตือนเมื่อค่าต่าง ๆ อยู่ในช่วงที่ไม่เหมาะสม ผลลัพธ์ของโครงการนี้คือการพัฒนา Web Application ที่ช่วยให้ผู้ใช้งานสามารถเข้าใจและติดตามสภาพแวดล้อมของพืชได้อย่างมีประสิทธิภาพ และเป็นตัวอย่างของการประยุกต์ใช้เทคโนโลยี Web Development ร่วมกับ 3D Visualization และข้อมูลจากระบบภายนอกได้อย่างเหมาะสม

วัตถุประสงค์และขอบเขต (Objectives & Scope):

- เพื่อพัฒนา Web Application ที่สามารถแสดงผลข้อมูลสภาพแวดล้อมของพืชในรูปแบบ Dashboard ที่เข้าใจง่ายและเป็นมิตรกับผู้ใช้
- เพื่อออกแบบและพัฒนา Frontend Interface ที่มีความสามารถในการโต้ตอบกับผู้ใช้ (Interactive UI) เช่น การแสดงผลข้อมูลแบบเรียลไทม์ กราฟ และสถานะของพืช
- เพื่อพัฒนา Backend System สำหรับจัดการข้อมูล (Data Management) รวมถึงการรับ ส่ง และประมวลผลข้อมูลจากแหล่งข้อมูลภายนอก เช่น mock data
- เพื่อออกแบบระบบแจ้งเตือน (Notification System) และการแสดงผลคำแนะนำในรูปแบบ Web-based Interaction
- เพื่อศึกษาการพัฒนา Web Application แบบ Full-stack ที่เชื่อมต่อระหว่าง Frontend, Backend และ Data Source

2. สถาปัตยกรรมระบบ (System Architecture)



2.1 . ภาพรวมระบบ (System Overview)

ระบบถูกออกแบบมาให้เป็น End-to-End IoT Solution ตั้งแต่การรับค่าจากเซ็นเซอร์ในสวน ไปจนถึงการประมวลผลด้วย AI และแสดงผลบน Dashboard โดยมีลำดับการทำงาน (Data Flow) ดังนี้

1. Hardware Layer: กิ่งและเซ็นเซอร์ (ความชื้น, อุณหภูมิ) ส่งข้อมูลผ่านโปรโตคอล (เช่น MQTT หรือ HTTP) เข้าสู่ส่วนประมวลผล
2. AI/Processing Layer: ใช้ Python (FastAPI/Flask) จัดการภาพจากกล้องเพื่อวิเคราะห์สุขภาพพืช หรือประมวลผลข้อมูลดิบ
3. Backend API: ทำหน้าที่เป็นตัวกลาง (Orchestrator) จัดการ Business Logic, ระบบสมาชิก และเชื่อมต่อกับภายนอก (Open-Meteo API)
4. Database: เก็บข้อมูลประวัติย้อนหลัง (Time-series data) และโครงสร้างระบบ
5. Frontend: Next.js ดึงข้อมูลจาก API มาแสดงผลในรูปแบบ Dashboard ที่ตอบสนองไว

ส่วนของงาน	เทคโนโลยีที่เลือกใช้	เหตุผลในการเลือกใช้
Frontend	Next.js (App Router)	รองรับ Server-Side Rendering (SSR) ทำให้การโหลดข้อมูลพยากรณ์อากาศจาก Open-Meteo รวดเร็วและ SEO ดี
Styling	Tailwind CSS	ช่วยให้เขียนสไตล์ได้ไว ควบคุม Design System ได้ง่ายผ่าน Utility-first classes

Design Pattern	Atomic Design	เพื่อแยก Component เป็นส่วนย่อย (Atoms, Molecules, etc.) ทำให้ Reusable สูง และจัดการ Dashboard ที่ซับซ้อนได้เป็นระเบียบ
Backend	Node.js	ประสิทธิภาพสูงในการจัดการ I/O และเข้ากับ Next.js ได้อย่างไร้รอยต่อ
AI Engine	Python (FastAPI)	มี Library ด้าน Data Science และ Image Processing (OpenCV/PyTorch)
Database	PostgreSQL	เป็น Relational Database ที่เสถียร และรองรับ TimescaleDB extension สำหรับเก็บข้อมูลเซ็นเซอร์ปริมาณมากได้ดี

สำหรับการทำ Monitoring เราจะใช้แนวคิด Atomic Design ในการสร้าง Dashboard ดังนี้

- 1. Atoms: ปุ่มกด, ไอคอนสภาพอากาศ, ตัวเลขแสดงอุณหภูมิ
- 2. Molecules: การ์ดแสดงสถานะเซ็นเซอร์ 1 ตัว, กราฟเส้นขนาดเล็ก (Sparkline)
- 3. Organisms: แผงควบคุมการรดน้ำ, ส่วนแสดงผลพยากรณ์อากาศ 7 วัน (ดึงจาก Open-Meteo)
- 4. Templates/Pages: หน้า Dashboard รวมที่นำ Organisms มาจัดวางแบบ Responsive

2.2. การออกแบบเส้น API (API Endpoints Design)

เพื่อให้ระบบทำงานได้สมบูรณ์ Backend API (Node.js) จะต้องทำหน้าที่เป็นศูนย์กลางในการจัดการข้อมูลทั้งหมด โดยแบ่งกลุ่ม API ตามหน้าที่ดังนี้ครับ

2.2.1 ส่วน AI/Python Processing Layer (ส่งข้อมูลจาก Hardware)

ส่วนนี้เป็น API ที่ FastAPI เรียกใช้เพื่อส่งผลการวิเคราะห์

Endpoint Path	Method	Description	Request Body (Example)
/api/hardware/data	POST	Hardware ส่งข้อมูล เซ็นเซอร์	{ "device_id": "SN-001", "temp": 25.5, "humidity": 60, "timestamp": "..." }
/api/ai/analysis	POST	AI Engine ส่งผล วิเคราะห์	{ "device_id": "SN-001", "image_url": "s3/path/plant.jpg", "health_status": "healthy", "diagnose": "normal growth", "timestamp": "..." }

2.2.2 Frontend Monitoring

API ที่ Next.js (Frontend) เรียกใช้เพื่อนำข้อมูลมาแสดงใน Dashboard

Endpoint Path	Method	Description	Response Body (Example)
---------------	--------	-------------	-------------------------

/api/dashboard/summary	GET	ดึงข้อมูลภาพรวมล่าสุด	{ "current_temp": 26, "latest_health": "normal", "weather_forecast": { ... } }
/api/{plant_type}/{plant_id}	GET	ดึงรายละเอียดพืช 1 ต้น	{ "id": 1, "name": "Monstera", "health_history": [...], "3d_model_url": "..." }
/api/sensors/{plant_type}/history	GET	ดึงข้อมูลย้อนหลัง	{ "labels": ["10:00", "11:00"], "datasets": [{ "label": "Temp", "data": [25, 26] }] }

2.2.3 Dashboard Control

API สำหรับใช้สั่งการ เช่น รดน้ำต้นไม้

Endpoint Path	Method	Description	Request Body (Example)
/api/controls/watering	POST	สั่งเปิด/ปิดระบบรดน้ำ	{ "action": "start", "duration_minutes": 5 }
/api/settings	PUT	ตั้งค่าแจ้งเตือน	{ "min_humidity": 40, "max_temp": 30 }

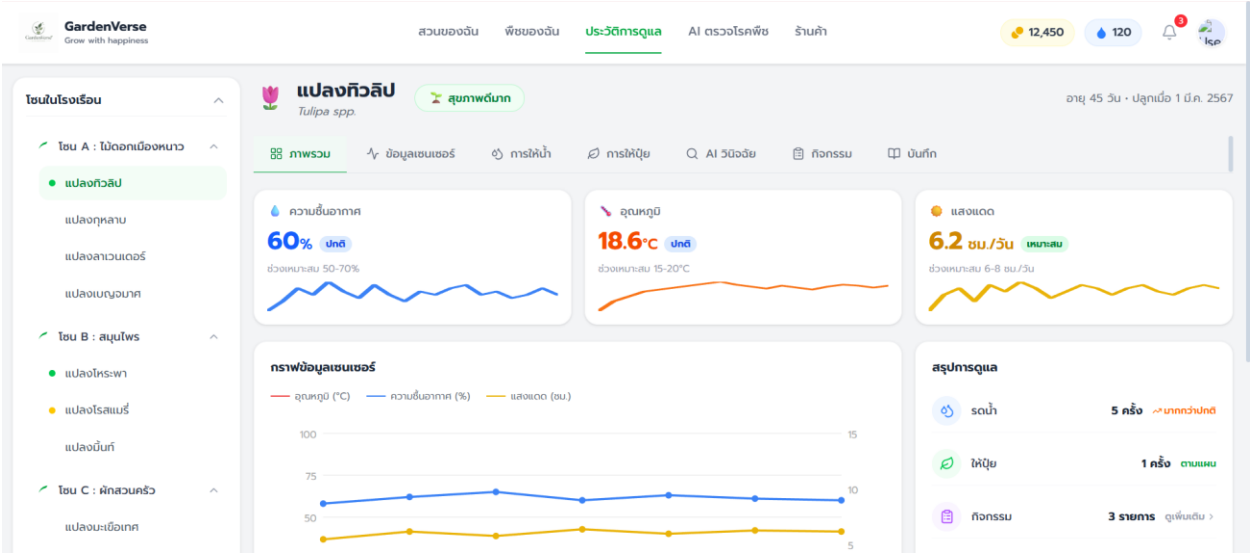
3. การพัฒนาและแก้ปัญหา (Implementation)

การพัฒนาฝั่ง Frontend & Backend: อธิบายโครงสร้าง API Route และวิธีการจัดการ State (Real-time update) บนหน้าเว็บ

การบูรณาการ Hardware/AI (Integration): อธิบายวิธีการดึงข้อมูลจาก Python/C++ เข้ามาที่ Web API (พร้อมยกตัวอย่าง Payload JSON ที่ส่ง)

ความปลอดภัย (Security): การจัดการ API Key หรือการตั้งค่า CORS ป้องกันการเข้าถึงที่ไม่ได้รับอนุญาต

4. ผลการทดสอบ (Results & Testing)



ภาพรวมหน้าจอ (UI/UX Showcase): ภาพ Screenshot ของหน้าเว็บในสถานะต่างๆ (เช่น หน้า Dashboard ปกติ, หน้าจอแจ้งเตือนเมื่อเซ็นเซอร์ผิดปกติ)

ผลการทดสอบระบบ (Performance & Latency): ความหน่วงในการส่งข้อมูลจากหุ่นยนต์มาแสดงผลบนเว็บ หรือความแม่นยำของ AI

ปัญหาที่พบและวิธีแก้ไข (Challenges & Solutions): (เช่น เจอปัญหา CORS Blocked และแก้ด้วยการปรับ next.config.mjs)

5. บทสรุปและข้อเสนอแนะ (Conclusion & Future Work)

5.1 บทสรุป

5.2 ข้อเสนอแนะ

สรุปความสำเร็จเทียบกับวัตถุประสงค์

แนวทางการนำไปใช้งานจริงในโรงงานอุตสาหกรรม หรือการอัปเกรดระบบในอนาคต

6. ลิงค์ Project และ Source code

ลิงค์: <https://github.com/chanyapakSub/GardenVerse>

7. รายชื่อสมาชิกและหน้าที่ที่รับผิดชอบ

ชื่อ – นามสกุล	ความรับผิดชอบในทีม
ณัฏฐา เลิศมหากุล	UX/UI Design, Backend
นันท์นภัส นันทพรนิรชา	UX/UI Design, Backend
อนวัช อนุสุเรนทร์	Frontend, Backend
ชญญักษ์ ทรัพย์สวัสดิ์กุล	Frontend, Backend

8. การใช้งานเครื่องมือปัญญาประดิษฐ์ (AI Usage Declaration)

"รายวิชานี้อนุญาตและส่งเสริมให้ใช้เครื่องมือ AI เป็นผู้ช่วยในการเขียนโค้ดและแก้ไขบั๊ก (Debugging) แต่ นักศึกษาจะต้องเป็นผู้รับผิดชอบต่อโค้ดทุกบรรทัดที่อยู่ในโปรเจกต์ หากมีการตรวจพบว่าโค้ดส่วนใดเกิดข้อผิดพลาด หรือนักศึกษาไม่สามารถอธิบายการทำงานของโค้ดที่ AI สร้างขึ้นมาระหว่างการสอบนำเสนอ

กลุ่มของข้าพเจ้าขอรับรองว่า ในการจัดทำโครงงานนี้ มีการใช้งานเครื่องมือ AI (เช่น ChatGPT, Gemini, GitHub Copilot, Claude ฯลฯ) เพื่อช่วยในการพัฒนา โดยมีรายละเอียดความโปร่งใสดังต่อไปนี้:

1. เครื่องมือ AI ที่ใช้งาน (AI Tools Used):

(เช่น ChatGPT (GPT-4), Gemini Advanced, GitHub Copilot)

2. วัตถุประสงค์และขอบเขตการใช้งาน (Purpose & Extent of Use): (ให้ระบุให้ชัดเจนว่าใช้ AI ช่วยในส่วนใดบ้าง)

- สร้างโครงร่างโปรเจกต์ / โค้ดพื้นฐาน (Boilerplate): (อธิบายเพิ่มเติม เช่น ใช้ AI ช่วยขึ้น โครงสร้างโฟลเดอร์ SvelteKit หรือ Next.js API เบื้องต้น)
- เขียนลอจิก / อัลกอริทึม (Logic Generation): (อธิบายเพิ่มเติม เช่น ใช้ AI ช่วยเขียนคำสั่ง SQL สำหรับดึงข้อมูลประวัติย้อนหลัง 20 แถว)
- ค้นหาและแก้ไขข้อผิดพลาด (Debugging): (อธิบายเพิ่มเติม เช่น ใช้ AI ช่วยวิเคราะห์สาเหตุของ Error CORS Policy หรือ บั๊กการเชื่อมต่อ PostgreSQL)

“การออกแบบและ CSS (UI/UX Design): (อธิบายเพิ่มเติม เช่น ให้ AI ช่วยเขียนคลาส Tailwind CSS สำหรับทำ Responsive Card)

“การเขียนรายงาน (Documentation): (อธิบายเพิ่มเติม เช่น ใช้ AI ช่วยเรียบเรียงไวยากรณ์ในบทคัดย่อ)

3. การตรวจสอบและรับรองความถูกต้อง (Human Validation & Accountability): (กลุ่มของข้าพเจ้าได้ทำการตรวจสอบโค้ดหรือเนื้อหาที่ AI สร้างขึ้นอย่างไร เพื่อให้มั่นใจว่าระบบทำงานได้จริงและปลอดภัย)

(ตัวอย่างการตอบ: "โค้ด API สำหรับรับค่าจากเซ็นเซอร์ที่ AI เจนเนอเรตมาในตอนแรกไม่รองรับการทำงานแบบ Real-time ทางกลุ่มจึงได้ทำการศึกษาเพิ่มเติมและแก้ไขโค้ดด้วยตนเองโดยเพิ่มระบบ Polling เข้าไป นอกจากนี้ได้ทดสอบยิงข้อมูลจริงจากบอร์ด ESP32 เพื่อยืนยันว่าข้อมูลลง Database ได้อย่างถูกต้อง"

"ทางกลุ่มใช้ GitHub Copilot ช่วยในการ Auto-complete โค้ด HTML และ CSS ของหน้า Dashboard และใช้ Gemini ในการช่วยหาวิธีแก้ Error 500 ตอนที่เชื่อมต่อ PostgreSQL โดย Gemini แนะนำให้เช็ค Type Mismatch ของตัวแปรทศนิยม ซึ่งทางกลุ่มได้นำมาวิเคราะห์และปรับแก้ค่าใน Database เป็น Float ด้วยตนเองจนระบบทำงานได้")